Reverse Image Search Project Documentation

CS370-002

Group 4

Michael Daley

Nicolas Camacho

Eric De La Rosa

SETTING UP ENVIRONMENT.

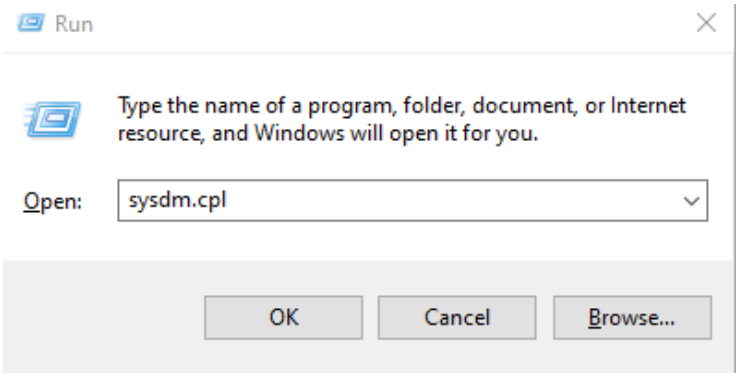
After some initial testing between utilizing AWS and personal machines due to shorter runtime on a local machine, our group decided to utilize a local machine to run our reverse image search algorithms. We also wanted to utilize Google Colab to allow for easy collaboration between members although one member would be doing most of the compilation of code and sharing results. Firstly, to allow for a connection between the local machine and Google Colab, Jupyter notebook had to be set up, as it is implemented by utilizing Colab. The local machine is a Windows machine, thus documentation will detail how to set up the environment on Windows. To install Jupyter notebook on a local machine, we first need to install the latest version of python from Download Python | Python.org as well as the python-pip installer from https://bootstrap.pypa.io/get-pip.py. This program was saved into a dedicated folder called python on the C:/ drive for ease of use. From here we can open the command line and change the directory into the C:/Python directory where our pip installer is, and we can pip install all the necessary programs and apps for our project; the first being Jupyter notebook, which we can just install with “pip install Jupyter notebook”. Afterward, we need to update Jupyter notebook to allow for it to connect to Google Colab with the joint commands: “pip install --upgrade jupyter_http_over_ws>=0.0.7 && Jupyter serverextension enable --py jupyter_http_over_ws”. Next, when launching the Jupyter notebook we utilize the command “jupyter notebook --NotebookApp.allow_origin='https://colab.research.google.com' --port=8888 --NotebookApp.port_retries=0” which includes arguments that allow the notebook to connect to the Colab website and to connect through port 8888. Now our notebook is online and capable of connecting to our Google Colab, from here we simply need to head over to the Colab notebook and select the connect tab at the upper right corner choosing the local machine.

In the command window we used to launch the Jupyter notebook, we will see the address used to connect our Jupyter notebook to Colab that we copy and paste within the address section on Colab.

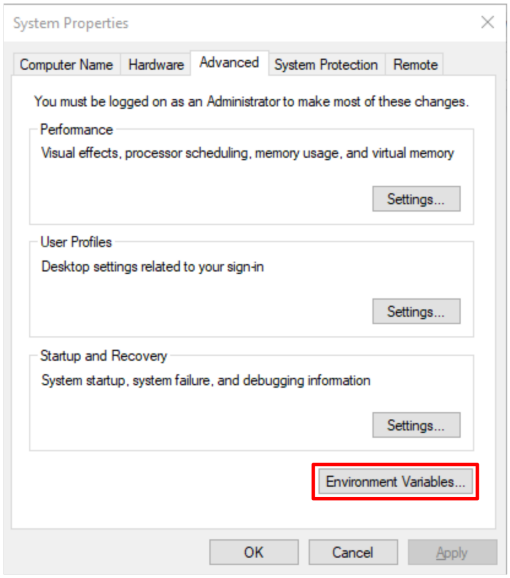
```
jupyter_http_over_ws extension initialized. Listening on /http_over_websocket
[I 22:22:51.649 NotebookApp] Serving notebooks from local directory: C:\python
[I 22:22:51.649 NotebookApp] Jupyter Notebook 6.4.11 is running at:
[I 22:22:51.649 NotebookApp] http://localhost:8888/?token=39ba7e33f2a6c693528193a2e02fb03e894d69cf2dd4e1d2
[I 22:22:51.649 NotebookApp]  or http://127.0.0.1:8888/?token=39ba7e33f2a6c693528193a2e02fb03e894d69cf2dd4e1d2
[I 22:22:51.649 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 22:22:51.696 NotebookApp]
```

Now that we are able to link Colab to our local machine and can easily run shared code, we need to install the necessary python libraries and modules to allow our local machine to run our program. For most of these, it will simply be pip install <name of lib>. In our project, the libraries that we utilized that needed to be installed were Keras, requests, numpy, joblib, pillow, tqdm, scikit-learn(sklearn), matplotlib, and lastly TensorFlow. The purpose of these will be explained later. Of the aforementioned, special attention was paid to the installation of TensorFlow as when we initially installed TensorFlow it didn't utilize the local machines gpu. Thus, further files and programs needed to be downloaded and adjusted. This section is optional as not everyone may have the appropriate hardware for it and the reverse image search would work without it, but in our case allowing TensorFlow to utilize our gpu (one that has Cuda architecture) in its computations exponentially improved performance. To enable this on the local machine we

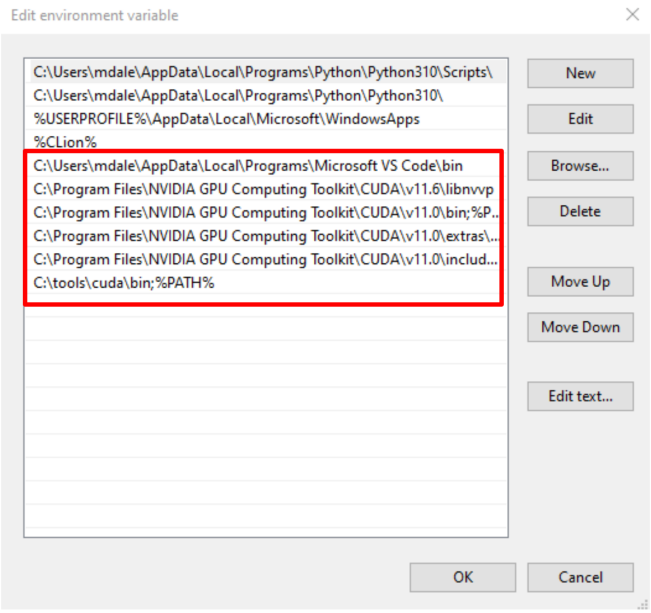
need to install the latest NVIDIA GPU DRIVERS from here https://www.nvidia.com/download/index.aspx?lang=en-us, install CUDA TOOLKIT from here http://developer.nvidia.com/cuda-downloads, and install cuDNN SDK from here https://developer.nvidia.com/rdp/cudnn-download. Once all of these have been properly downloaded we then need to extract the files contained within the cuDNN folder into the NVIDIA GPU Computing Toolkit (this folder comes from installing the CUDA TOOLKIT, so ensure this is done prior). NVIDIA GPU Computing Toolkit folder will be located in C:\Program Files if the default install location was chosen. Next, we need to copy the zlibwapi.dll(required for compression) from C:\Program Files\Microsoft Office\root\Office16\ODBC Drivers\Salesforce\lib and paste it into C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.6\bin assuming that the NVIDIA GPU Computing Toolkit folder is in the aforementioned location. Lastly, we need to add these three paths to our environment variable. Press Windows + R, type sysdm.cpl, and click “okay”.



Next, select the Advanced tab and then environment variables,



Click on “path” under user variables and then add these five paths separately. “C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.6\libnvvp”, “C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.0\bin;%PATH%”, “C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.0\extras\CUPTI\lib64;%PATH%”, “C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.0\include;%PATH%”, “C:\tools\cuda\bin;%PATH%”.



With that, the environment should be fully set up and able to run any code in the following sections.

SETTING UP IMPORTS.

```
import gc
from tqdm import tqdm
import os
import math
import PIL
import requests
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import tarfile
import numpy as np
from numpy.linalg import norm
import joblib as pickle
import time
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
```

As we mentioned before in the Setting Up Environment section, our group needed to import and install several libraries to get started with the reverse image searching. The most important library we needed was TensorFlow, which is a free software library for machine learning. The TensorFlow library will be a great help in processing the images and generating a feature list for our model. The rest of the imports such as “numpy”, “math”, and “matplotlib” are going to be used to plot the data that is generated from the predictions that the machine makes.

DOWNLOADING AND EXTRACTING THE DATA FROM LFW.

Download the Data from LFW

```
[ ] downloadUrl = 'http://vis-www.cs.umass.edu/lfw/lfw.tgz'
    request = requests.get(downloadUrl)
    fileName = 'lfw.tgz'
    folderName = 'lfw'
    open(fileName , 'wb').write(request.content)
```

```
180566744
```

extract data from downloaded lfw.tgz file

```
[ ] # reference https://stackoverflow.com/questions/30887979/i-want-to-create-a-script-for-unzip-tar-gz-file-via-python
    inputFile = tarfile.open(fileName, "r:gz")
    inputFile.extractall()
    inputFile.close()
```

The first block of the code takes the URL shown and immediately requests to download the data from it. The data is saved in a file named “lfw.tgz” and it exists under the folder “lfw.” The second block of code then extracts all the data from that “lfw.tgz” file and then closes the file.

IMAGE PROCESSING/DATA GENERATION AND CREATING A FEATURE LIST OF THE MODEL.

```python
imgSize = 224

# The following is a Resnet50 model without the top layers so we get convolutional features as output instead of the image classification probability.

model = ResNet50(weights='imagenet', include_top=False,input_shape=(imgSize, imgSize, 3),pooling='max')

# Create the keras image data generator object and extract the features for the images using Resnet50 predict_generator method.

batchSize = 64
dir = 'lfw'

# reference https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator
# Generate batches of tensor image data with real-time data augmentation.

imageGenerator = ImageDataGenerator(preprocessing_function=preprocess_input)

# reference https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator#flow_from_directory
# Takes the path to a directory & generates batches of augmented data.

dataGenerator = imageGenerator.flow_from_directory(dir, target_size=(imgSize, imgSize), batch_size=batchSize, class_mode=None, shuffle=False)

# Number of images and amount of epochs associated with the batch size

numImages = len(dataGenerator.filenames)
numEpochs = int(math.ceil(numImages / batchSize))

# reference https://www.tensorflow.org/api_docs/python/tf/keras/Model#predict
# Generates output predictions for the input samples.

featureList = model.predict(dataGenerator, numEpochs)

# We have 13233 images in our dataset. Each image is converted into a 2048 feature vector

print("Num images   = ", len(dataGenerator.classes))
print("Shape of feature list = ", featureList.shape)

# Get full path for all the images in our dataset

fileNames = [dir + '/' + s for s in dataGenerator.filenames]

# We will fit the nearest neighbor algorithm to the extracted features. We have picked kd_tree algorithm

neighbors = NearestNeighbors(n_neighbors=20, algorithm='kd_tree', metric='euclidean')
neighbors.fit(featureList)
```

We used ResNet to create a model that has the top layers removed. Leaving only the convolutional features instead of the image classification probabilities. We then create a Keras image data generator object using a batch size of 64 to extract the features from the images using a ResNet prediction generator. The term “batch size” refers to the number of samples that will be propagated through the network. We initialized the number of images to 20 and then created the number of epochs by dividing the number of images by the batch size. The term “epoch” is used to describe one full training cycle of the algorithm on the training set. Once all the samples have been seen by the network the next “epoch” or “full training cycle” will begin. We then ran the model through a prediction generator to create a feature list. We then output the number of images in the dataset and the shape of the feature list. After creating a list of the filenames for each of the images to iterate through, we will fit the features to the kd tree nearest neighbors algorithm. A k-d tree is a binary tree structure where each node is a k-dimensional point. The nearest neighbor search (NN) algorithm aims to find the point in the tree that is nearest to a given input point. This search can be done efficiently by using the tree properties to quickly eliminate large portions of the search space.

PLOTTING THE SIMILAR IMAGES.

```
[41]
def similar_images(indices):
    plt.figure(figsize=(15,10), facecolor='none')
    plotNum = 1
    for index in indices:
        if plotNum<=len(indices) :
            ax = plt.subplot(5,4,plotNum)
            plt.xlabel('Similar Image '+ str(plotNum),fontsize= 20,color = 'black')
            plt.xticks([])
            plt.yticks([])
            plt.imshow(mpimg.imread(fileNames[index]), interpolation='lanczos')
            plotNum+=1
    plt.tight_layout()
```

This function is used to plot out the similar images from the list of predicted images the model accumulated. It first creates a figure of dimensions 15 by 10 and then loops through the array of photos placing them into a 5 by 4 grid and labeling each image.

PREDICTING SIMILAR IMAGES.

```
[42]
def prediction(path, url):
  image_name = path.split('.')[0]
  numberOfImages = 20
  data = requests.get(url)
  open(path, 'wb').write(data.content)

  # ref https://datascience.stackexchange.com/questions/31167/how-to-predict-an-image-using-saved-model
  input_shape = (imgSize, imgSize, 3)
  img = image.load_img(path, target_size=(input_shape[0], input_shape[1]))
  imageArray = image.img_to_array(img)
  expandedImageAarray = np.expand_dims(imageArray, axis=0)
  preprocessedImage = preprocess_input(expandedImageAarray)

  # Make model predictions from preprocessed image
  testImageFeatures = model.predict(preprocessedImage, batch_size=1)
  indices = neighbors.kneighbors(testImageFeatures)

  # Use Prinicipal Component Analyusis To reduce dimensionality and get more
  # accurate predicitons with nearest neighbors algorithm
  pca = PCA(n_components=100)
  pca.fit(featureList)
  compressed_features = pca.transform(featureList)

  neighborsFeatures = NearestNeighbors(n_neighbors= numberOfImages, algorithm='ball_tree', metric='euclidean').fit(compressed_featu

  testImageCompressed = pca.transform(testImageFeatures)
  distances, indices = neighborsFeatures.kneighbors(testImageCompressed)

  # PLot the original image
  plt.imshow(mpimg.imread(path), interpolation='lanczos')
  plt.xlabel(image_name + ' input Image',fontsize= 20,color = 'black')
  plt.xticks([])
  plt.yticks([])
  plt.show()

  # plot the predictions
  print('PREDICTIONS'.rjust(70))
  similar_images(indices[0])
```

The prediction function is a driver function for actually predicting similar images from the original image. We first initialize the input shape. This shape is a 224 by 224 shape with 3 channels for red, green, and blue. We then load the individual image and convert it into an array which is later padded by expanding its dimensions of it. We then preprocess the image to later be used in the model predictions. We use the preprocessed images to get the predicted features from the ResNet model. We then use a K-nearest neighbor algorithm from sklearn to generate a list of the indices of the nearest neighbors to the point. We then use a Principal Component Analysis to reduce the dimensionality of the feature list so we can get more precise predictions. We fit the feature list into a PCA and then transform it to get a list of compressed features. We then use a ball tree nearest neighbor algorithm on the compressed images to get the features of the neighbors. A “Ball Tree” is a special data structure that attempts to partition the training data in such a way that only a portion of the training data has to be searched. We then use the PCA to compress the test image features and use these compressed image features to get a list of nearest neighbor distances and indices. We then plot the original image and then call on the similar images functions to plot out the images in the indices sent as a parameter.

DRIVER FUNCTION.

```
def driver(name): #persons name
  path = name + '.jpg'
  url = 'https://raw.githubusercontent.com/MDaley08/CS-370-Project/main/' + path
  prediction(path, url)
```

The driver function creates the proper path and URL for the main images on the Github page and calls the prediction function with the appropriate path and URL parameters.